

Operating Systems: Assignment-13

Name: Suraj Shah

Admission Number: I24AI012

Q 1. Write a program to simulate segmentation using a static segment table and perform address translation.

Code:

```
#include <stdio.h>

#define MAX_SEGMENTS 10

struct Segment {
    int base;
    int limit;
};

int main() {
    struct Segment segTable[MAX_SEGMENTS];
    int n, i;

    printf("Enter number of segments: ");
    scanf("%d", &n);

    printf("Enter base and limit for each segment:\n");
    for (i=0; i<n; i++) {
        printf("Segment %d:\n", i);
        printf("Base: ");
        scanf("%d", &segTable[i].base);
        printf("Limit: ");
        scanf("%d", &segTable[i].limit);
    }

    int seg_no, offset;

    while (1) {
        printf("\n Enter segment number (-1 to exit): ");
        scanf("%d", &seg_no);

        if (seg_no== -1) {
            break;
        }

        printf("Enter offset: ");
```

```

scanf("%d", &offset);

if (seg_no>=n || seg_no<0) {
    printf("Invalid segment number!\n");
    continue;
}

if (offset>=segTable[seg_no].limit) {
    printf("Segmentation Fault! Offset exceeds limit.\n");
}
else {
    int physical_address=segTable[seg_no].base+offset;
    printf("Physical Address=%d\n", physical_address);
}
}
return 0;
}

```

Output:

```

Enter number of segments: 3
Enter base and limit for each segment:
Segment 0:
Base: 1000
Limit: 500
Segment 1:
Base: 2000
Limit: 300
Segment 2:
Base: 3000
Limit: 400

Enter segment number (-1 to exit): 1
Enter offset: 100
Physical Address=2100

Enter segment number (-1 to exit): 2
Enter offset: 500
Segmentation Fault! Offset exceeds limit.

Enter segment number (-1 to exit): -1

```

Q 2. Create a simulation that takes a logical address (segment number + offset) and translates it to a physical address using a segment table.

Code:

```
#include <stdio.h>

struct Segment {
    int base;
    int limit;
};

int main() {
    int n;

    printf("Enter number of segments: ");
    scanf("%d", &n);

    struct Segment segTable[n];

    printf("\nEnter base and limit for each segment:\n");
    for(int i=0; i<n; i++) {
        printf("Segment %d:\n", i);
        printf("Base: ");
        scanf("%d", &segTable[i].base);
        printf("Limit: ");
        scanf("%d", &segTable[i].limit);
    }

    int seg_no, offset;

    printf("\nEnter logical address:\n");
    printf("Segment number: ");
    scanf("%d", &seg_no);

    printf("Offset: ");
    scanf("%d", &offset);

    if(seg_no<0 || seg_no>=n) {
        printf("\nInvalid Segment Number!\n");
    }
    else if(offset>=segTable[seg_no].limit) {
        printf("\nSegmentation Fault! Offset exceeds limit.\n");
    }
    else {
        int physical_address=segTable[seg_no].base+offset;
        printf("\nPhysical Address=%d\n", physical_address);
    }
}
```

```
    return 0;  
}
```

Output:

```
Enter number of segments: 2  
  
Enter base and limit for each segment:  
Segment 0:  
Base: 1000  
Limit: 300  
Segment 1:  
Base: 2000  
Limit: 400  
  
Enter logical address:  
Segment number: 1  
Offset: 150  
  
Physical Address=2150
```

Q 3. Implement a model where each process is divided into segments (code, data, stack), and manage memory allocation and deallocation for each.

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_PROCESSES 10

struct Segment {
    int base;
    int limit;
    int allocated;
};

struct Process {
    int pid;
    struct Segment code;
    struct Segment data;
    struct Segment stack;
};

struct Process processes[MAX_PROCESSES];
int process_count=0;
int memory_pointer=1000;

void allocate_segment(struct Segment *seg, int size) {
    seg->base=memory_pointer;
    seg->limit=size;
    seg->allocated=1;
    memory_pointer+=size;
}

void create_process() {
    if (process_count>=MAX_PROCESSES) {
        printf("Process limit reached!\n");
        return;
    }

    struct Process p;
    p.pid=process_count;

    int code_size, data_size, stack_size;

    printf("\nEnter sizes for Process %d:\n", p.pid);
    printf("Code size: ");
    scanf("%d", &code_size);
    printf("Data size: ");
```

```

scanf("%d", &data_size);
printf("Stack size: ");
scanf("%d", &stack_size);

allocate_segment(&p.code, code_size);
allocate_segment(&p.data, data_size);
allocate_segment(&p.stack, stack_size);

processes[process_count++]=p;

printf("Process %d allocated successfully");
}

void delete_process() {
    int pid;
    printf("Enter process ID to delete: ");
    scanf("%d", &pid);

    if (pid<0 || pid>=process_count) {
        printf("Invalid PID!\n");
        return;
    }

    processes[pid].code.allocated=0;
    processes[pid].data.allocated=0;
    processes[pid].stack.allocated=0;

    printf("Process %d deallocated!\n", pid);
}

void display() {
    printf("\nProcess Table\n");

    for(int i=0; i<process_count; i++) {
        struct Process p=processes[i];

        printf("\nProcess %d:\n", p.pid);

        printf("Code -> Base: %d, Limit: %d, %s\n", p.code.base,
p.code.limit, p.code.allocated ? "Allocated" : "Free");

        printf("Data -> Base: %d, Limit: %d, %s\n", p.data.base,
p.data.limit, p.data.allocated ? "Allocated" : "Free");

        printf("Stack -> Base: %d, Limit: %d, %s\n", p.stack.base,
p.stack.limit, p.stack.allocated ? "Allocated" : "Free");
    }
}

```

```
int main() {
    int choice;

    while(1) {
        printf("\n1. Create Process\n2. Delete Process\n3. Display\n4.
Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                create_process();
                break;
            case 2:
                delete_process();
                break;
            case 3:
                display();
                break;
            case 4:
                exit(0);
            default:
                printf("Invalid choice!\n");
        }
    }

    return 0;
}
```

Output:

```
1. Create Process
2. Delete Process
3. Display
4. Exit
Enter choice: 1

Enter sizes for Process 0:
Code size: 200
Data size: 150
Stack size: 100
Process 100 allocated successfully
1. Create Process
2. Delete Process
3. Display
4. Exit
Enter choice: 3

Process Table

Process 0:
Code -> Base: 1000, Limit: 200, Allocated
Data -> Base: 1200, Limit: 150, Allocated
Stack -> Base: 1350, Limit: 100, Allocated

1. Create Process
2. Delete Process
3. Display
4. Exit
Enter choice: 2
Enter process ID to delete: 0
Process 0 deallocated!

1. Create Process
2. Delete Process
3. Display
4. Exit
Enter choice: 3

Process Table

Process 0:
Code -> Base: 1000, Limit: 200, Free
Data -> Base: 1200, Limit: 150, Free
Stack -> Base: 1350, Limit: 100, Free
```